UNIT-V

DEEP LEARNING

UNIT – V

BASICS OF DEEP LEARNING

- Deep learning architectures: Convolutional Neural Networks : Neurons in Human Vision
- The Shortcomings of Feature Selection
- Full Description of the Convolutional Layer
- Max Pooling
- Full Architectural Description of Convolution Networks
- Closing the Loop on MNIST with Convolutional Networks
- Building a Convolutional Network for CIFAR-10
- Visualizing Learning in Convolutional Networks
- Leveraging Convolutional Filters to Replicate Artistic Styles
- Learning Convolutional Filters for Other Problem Domains
- Training algorithms.

Deep learning architectures: Convolutional Neural Networks : Neurons in Human Vision

- The human sense of vision is unbelievably advanced. Within fractions of seconds, we can identify objects within our field of view, without thought or hesitation.
- Not only can we name objects we are looking at, we can also perceive their depth, perfectly distinguish their contours, and separate the objects from their backgrounds.
- Somehow our eyes take in raw voxels of color data, but our brain transforms that information into more meaningful primitives—lines, curves, and shapes—that might indicate, for example, that we're looking at a house cat.
- Foundational to the human sense of vision is the neuron.
- Specialized neurons are responsible for capturing light information in the human eye.
- This light information is then preprocessed, transported to the visual cortex of the brain, and then finally analyzed to completion.
- Neurons are single-handedly responsible for all of these functions.
- As a result, intuitively, it would make a lot of sense to extend our neural network models to build better computer vision systems.
- In this chapter, we will use our understanding of human vision to build effective deep learning models for image problems.
- But before we jump in, let's take a look at more traditional approaches to image analysis and why they fall short.

The Shortcomings of Feature Selection

- Considering a simple computer vision problem.
- A randomly selected image is given, such as the one in Figure 5-1.
- Task is to tell me if there is a human face in this picture.
- This is exactly the problem that Paul Viola and Michael Jones tackled in their seminal paper published in 2001.
- For a human like you or me, this task is completely trivial.
- For a computer, this is a very difficult problem.
- How do we teach a computer that an image contains a face?
- Try to train a traditional machine learning algorithm by giving it the raw pixel values of the image and hoping it can find an appropriate classifier.
- doesn't work well because the signal-to-noise ratio is much too low for any useful learning to occur

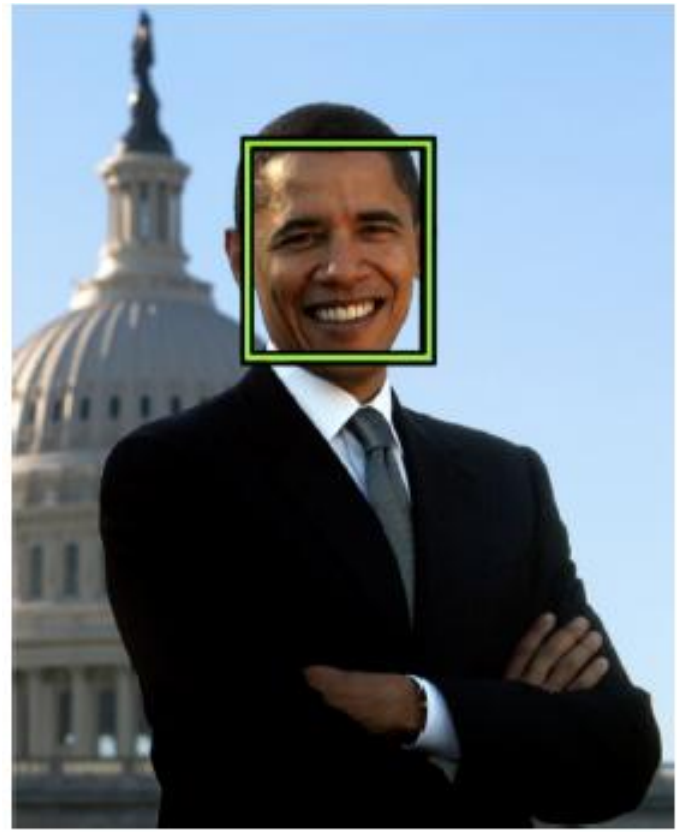


Figure 5-1. A hypothetical face-recognition algorithm should detect a face in this photograph of former President Barack Obama

- A human would choose the features (perhaps hundreds or thousands) that he or she believed were important in making a classification decision.
- The human would be producing a lower-dimensional representation of the same learning problem.
- The machine learning algorithm would then use these new *feature vectors to make classification decisions.*
- *Because the feature extraction process* improves the signal-to-noise ratio, this approach had quite a bit of success compared to the state of the art at the time.
- Viola and Jones had the insight that faces had certain patterns of light and dark patches that they could exploit.
- Ex: There is a difference in light intensity between the eye region and the upper cheeks.

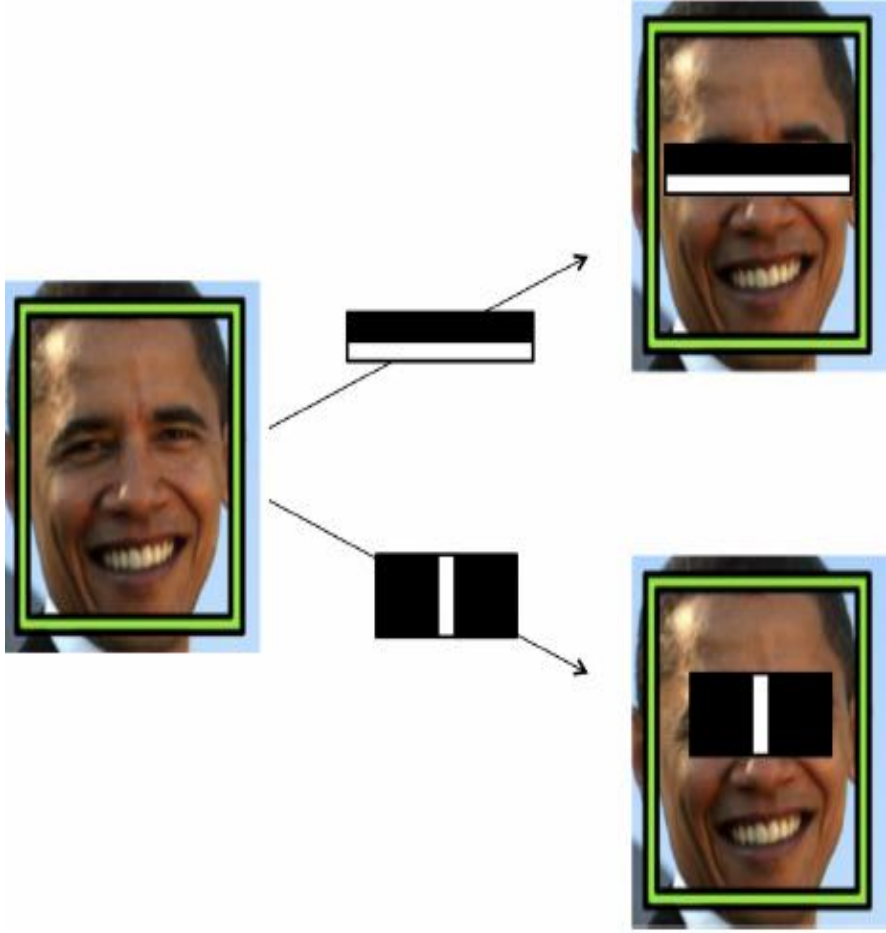


Figure 5-2. An illustration of Viola-Jones intensity detectors

There is also a difference in light intensity between the nose bridge and the two eyes on either side. These detectors are shown in Figure 5-2.

- By themselves, each of these features is not very effective at identifying a face.
- But when used together their combined effectiveness drastically increases.
- On a dataset of 130 images and 507 faces, the algorithm achieves a 91.4% detection rate with 50 false positives.

DEEP NEURAL NETWORKS

- The fundamental goal in applying deep learning to computer vision is to remove the cumbersome, and ultimately limiting, feature selection process.
- Deep neural networks are perfect for this process because each layer of a neural network is responsible for learning and building up features to represent the input data that it receives.
- MNIST dataset to achieve the image classification task.
- In MNIST, our images were only 28 x 28 pixels and were black and white.
- As a result, a neuron in a fully connected hidden layer would have 784 incoming weights.
- This technique, does not scale well as our images grow larger.
- Ex: For a full-color 200 x 200 pixel image, our input layer would have 200 x 200x 3 = 120,000 weights.

- The convolutional network takes advantage of the fact that we're analyzing images, and sensibly constrains the architecture of the deep network so that we drastically reduce the number of parameters in our model.
- Inspired by how human vision works, layers of a convolutional network have neurons arranged in three dimensions, so layers have a width, height, and depth, as shown in Figure 5-4.7
- A convolutional layer's function can be expressed simply: it processes a three-dimensional volume of information to produce a new three-dimensional volume of information.

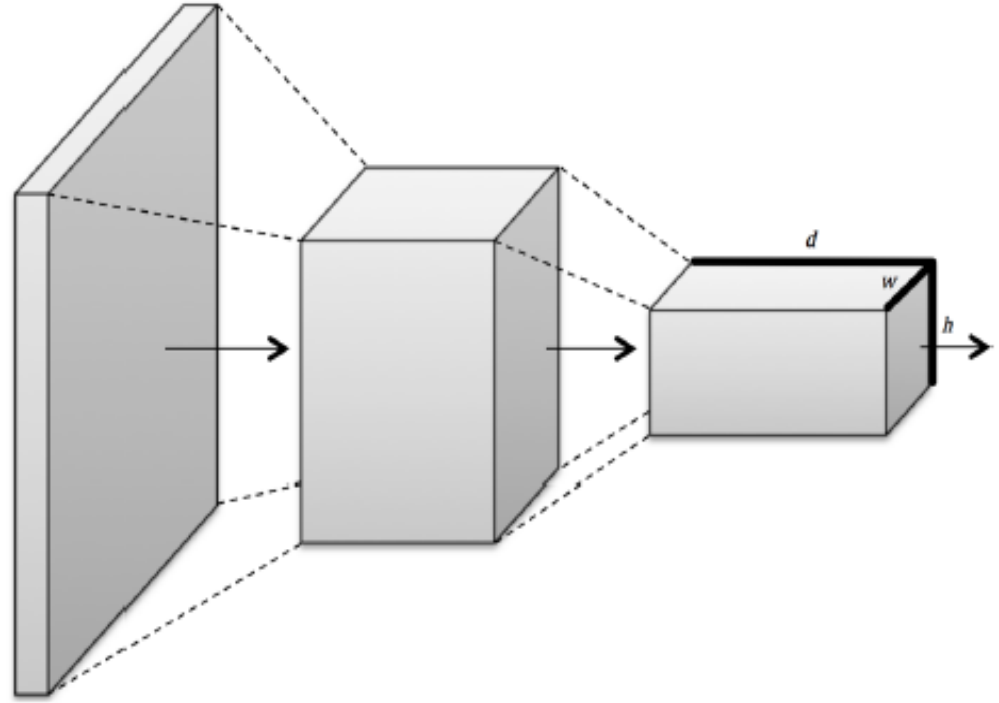


Figure 5-4. Convolutional layers arrange neurons in three dimensions, so layers have width, height, and depth

Full Description of the Convolutional Layer

- First, a convolutional layer takes in an input volume.
- This input volume has the following characteristics:
  - Its *width win*
  - Its *height hin*
  - Its *depth din*
  - Its *zero padding p*
- This volume is processed by a total of *k filters, which represent the weights and connections* in the convolutional network.
- These filters have a number of hyper parameters, which are described as follows:
  - Their *spatial extent e, which is equal to the filter's height and width.*
  - Their *stride s, or the distance between consecutive applications of the filter on the* input volume.
- If we use a stride of 1, we get the full convolution described in the previous section.
- We illustrate this in Figure 5-10.
  - The bias *b (a parameter learned like the values in the filter) which is added to* each component of the convolution.

- This results in an output volume with the following characteristics:
  - Its function f , which is applied to the incoming logit of each neuron in the output volume to determine its final value.
- The m^{th} "depth slice" of the output volume, where $1 \leq m \leq k$, corresponds to the function f applied to the sum of the m^{th} filter convoluted over the input volume and the bias b^m.
- This means that per filter, we have $d_{in}e^2$ parameters.

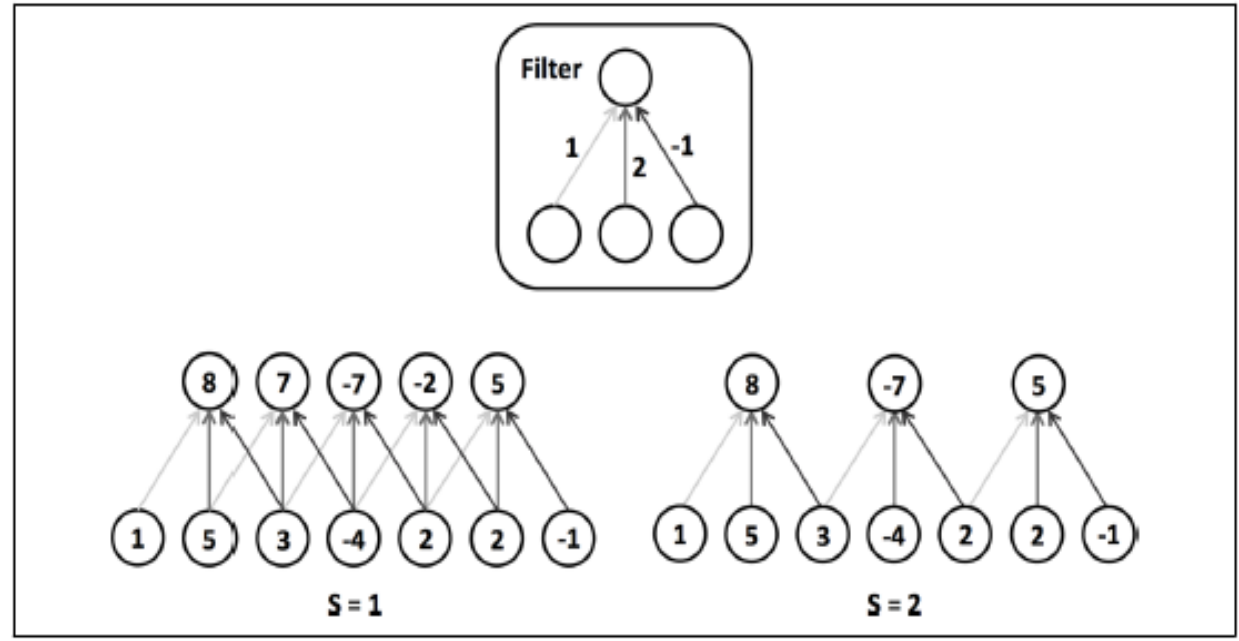


Figure 5-10. An illustration of a filter's stride hyperparameter

$$w_{out} = \left\lceil \frac{w_{in} - e + 2p}{s} \right\rceil + 1$$

- Its *height* $h_{out} = \left\lceil \frac{h_{in} - e + 2p}{s} \right\rceil + 1$
- Its *depth* $d_{out} = k$

The layer has $kd_{in}e^2$ *parameters and k biases.*

To demonstrate this in action, we provide an example of a convolutional layer in Figure 5-11 and Figure 5-12.

- with a 5 x 5 x 3 input volume with zero padding *p = 1. We'll use two 3 x 3 x 3 filters* (spatial extent) with a stride *s = 2.*
- *We'll use a linear function to produce the output* volume, which will be of size 3 x 3 x 2.

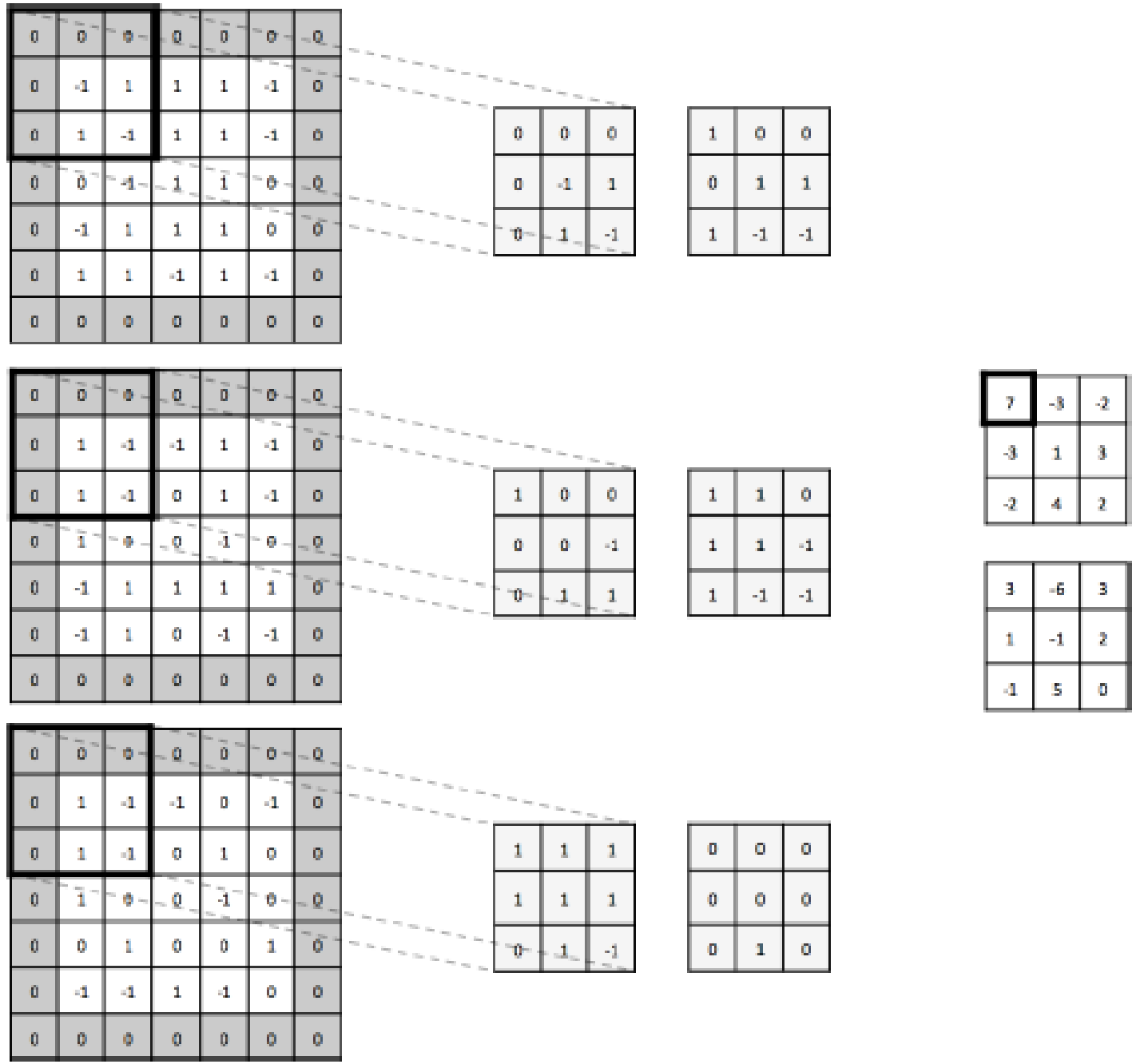


Figure 5-11. This is a convolutional layer with an input volume that has width 5, height 5, depth 3, and zero padding 1.
There are 2 filters, with spatial extent 3 and applied with a stride of 2.
It results in an output volume with width 3, height 3, and depth 2.
We apply the first convolutional filter to the upper-leftmost 3 x 3 piece of the input volume to generate the upper-leftmost entry of the first depth slice.

- Generally, it's wise to keep filter sizes small (size 3 x 3 or 5 x 5). Less commonly, larger sizes are used (7 x 7) but only in the first convolutional layer.
- Having more small filters is an easy way to achieve high representational power while also incurring a smaller number of parameters.
- It's also suggested to use a stride of 1 to capture all useful information in the feature maps, and a zero padding that keeps the output volume's height and width equivalent to the input volume's height and width.
- TensorFlow provides us with a convenient operation to easily perform a convolution on a minibatch of input volumes (note that we must apply our choice of function *f ourselves and it is not performed by the operation itself):*

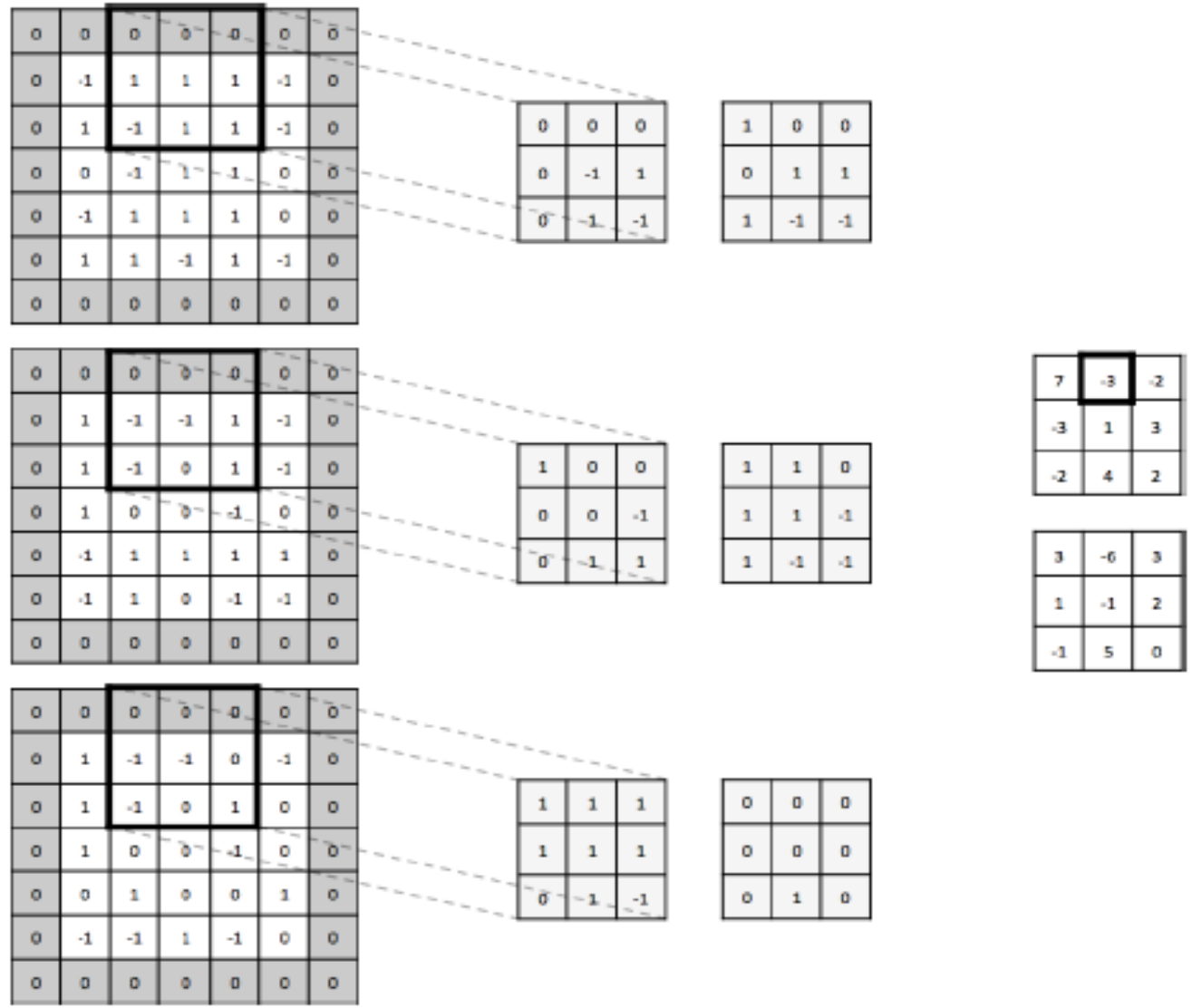


Figure 5-12. Using the same setup as Figure 5-11, we generate the next value in the first depth slice of the output volume.

Max Pooling

- To reduce dimensionality of feature maps and sharpen the located features, insert a *max pooling layer after a convolutional layer.*
- Essential idea behind max pooling is to break up each feature map into equally sized tiles.
- Create a condensed feature map.
- Create a cell for each tile, compute the maximum value in the tile, and propagate this maximum value into the corresponding cell of the condensed feature map.
- This process is illustrated in Figure 5-13.
- Describe a pooling layer with two parameters:
  - Its spatial extent *e*
  - Its stride *s*
- Only two major variations of the pooling layer are used.

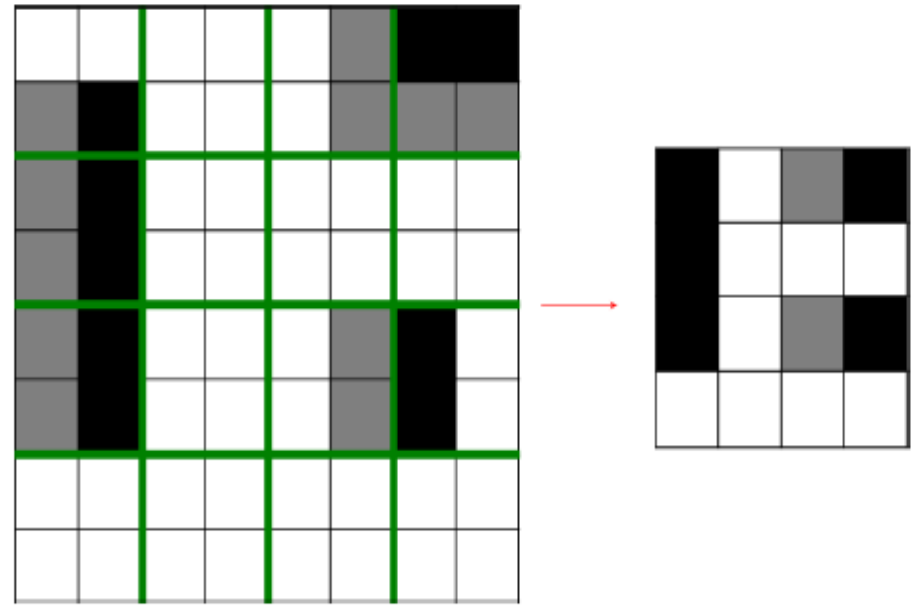


Figure 5-13. An illustration of how max pooling significantly reduces parameters as we move up the network

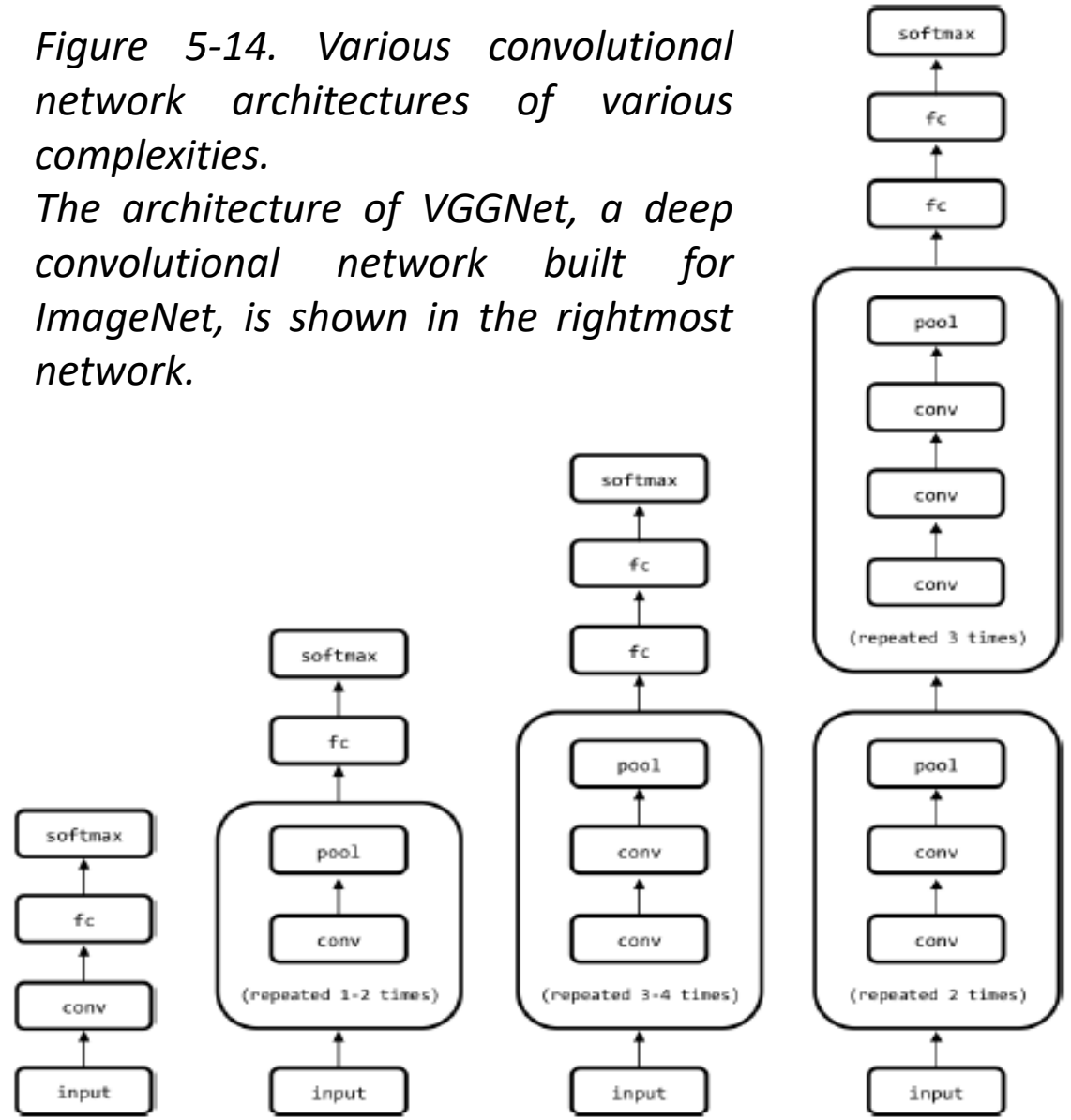
The first is the non-overlapping pooling layer with *e = 2, s = 2.*
The second is the overlapping pooling layer with *e = 3, s = 2.*
The resulting dimensions of each feature map are as follows:

- Its width $w_{out} = \left\lceil \frac{w_{in} - e}{s} \right\rceil + 1$
- Its height $h_{out} = \left\lceil \frac{h_{in} - e}{s} \right\rceil + 1$

Full Architectural Description of Convolution Networks

- Described the building blocks of convolutional networks, we start putting them together.
- Fig.5-14 depicts several architectures that might be of practical use.
- Build deeper networks is that we reduce the number of pooling layers and instead stack multiple convolutional layers in tandem.
- Helpful because pooling operations are inherently destructive.
- Stacking several convolutional layers before each pooling layer allows us to achieve richer representations.
- deep convolutional networks can take up a significant amount of
- space, and most casual practitioners are usually bottlenecked by the memory capacity on their GPU.

Figure 5-14. Various convolutional network architectures of various complexities.
The architecture of VGGNet, a deep convolutional network built for ImageNet, is shown in the rightmost network.



- Deep convolutional networks can take up a significant amount of space, and most casual practitioners are usually bottlenecked by the memory capacity on their GPU.
- The VGGNet architecture, for example, takes approximately 90 MB of memory on the forward pass per image and more than 180 MB of memory on the backward pass to update the parameters.
- Many deep networks make a compromise by using strides and spatial extents in the first convolutional layer that reduce the amount of information that needs to propagated up the network.

Closing the Loop on MNIST with Convolutional Networks

- Better understanding of how to build networks that effectively analyze images, revisit the MNIST challenge tackled.
- Use a convolutional network to learn how to recognize handwritten digits.
- Feed-forward network was able to achieve a 98.2% accuracy.
- Goal will be to push the envelope on this result.
- Build a convolutional network with a pretty standard architecture (modeled after the second network in Figure 5-14): two pooling and two convolutional interleaved, followed by a fully connected layer (with dropout, *p = 0 . 5) and a terminal softmax.*

- To make building the network easy, we write a couple of helper methods in addition to our layer generator from the feed-forward network:
- The first helper method generates a convolutional layer with a particular shape.
- Set the stride to be to be 1 and the padding to keep the width and height constant between input and output tensors.
- Initialize the weights using the same heuristic used in the feed-forward network.
- Number of incoming weights into a neuron spans the filter's height and width and the input tensor's depth.
- The second helper method generates a max pooling layer with non-overlapping windows of size k.
- The default, as recommended, is k=2, and we'll use this default in MNIST convolutional network.
- First take the flattened versions of the input pixel values and reshape them into a tensor of the *N × 28 × 28 × 1, where N is the* number of examples in a mini batch, 28 is the width and height of each image, and 1 is the depth (because the images are black and white; if the images were in RGB color, the depth would instead be 3 to represent each color map).

- We then build a convolutional layer with 32 filters that have spatial extent 5.
- This results in taking an input volume of depth 1 and emitting a output tensor of depth 32.
- This is then passed through a max pooling layer which compresses the information.
- Build a second convolutional layer with 64 filters, again with spatial extent 5, taking an input tensor of depth 32 and emitting an output tensor of depth 64.
- Again, passed through a max pooling layer to compress information.
- Prepare to pass the output of the max pooling layer into a fully connected layer.
- To do this, we flatten the tensor.
- Do this by computing the full size of each “subtensor” in the minibatch.
- Have 64 filters, which corresponds to the depth of 64.

Building a Convolutional Network for CIFAR-10

- The CIFAR-10 challenge consists of 32 x 32 color images that belong to one of 10 possible classes.
- This is a surprisingly hard challenge because it can be difficult for even a human to figure out what is in a picture.
- An example is shown in Figure 5-16.
- Build networks both with and without batch normalization as a basis of comparison.
- Increase the learning rate by 10-fold for the batch normalization network to take full advantage of its benefits.
- Only display code for the batch normalization network here because building the vanilla convolutional network is very similar.
- Distort random 24 × 24 crops of the input images to feed into our network for training.
- Use the example code provided by Google to do this.

Figure 5-16. A dog from the CIFAR-100 dataset

Integrate batch normalization into the convolutional and fully connected layers.

- Use two convolutional layers (each followed by a max pooling layer).
- There are then two fully connected layers followed by a softmax. Dropout is included for reference, but in the batch normalization version, keep_prob=1 during training:
- Finally, we use the Adam optimizer to train our convolutional networks.
- After some amount of time training, our networks are able to achieve an impressive 92.3% accuracy on the CIFAR-10 task without batch normalization and 96.7% accuracy with batch normalization.

Visualizing Learning in Convolutional Networks

- On a high level, the simplest thing that we can do to visualize training is plot the cost function and validation errors over time as training progresses.
- Demonstrate the benefits of batch normalization by comparing the rates of convergence between our two networks.
- Plots taken in the middle of the training process are shown in Figure 5-17.

- Without batch normalization, cracking the 90% accuracy threshold requires over 80,000 minibatches.
- On the other hand, with batch normalization, crossing the same threshold only requires slightly over 14,000 minibatches.
- Inspect the filters that our convolutional network learns in order to understand what the network finds important to its classification decisions.
- Convolutional layers learn hierarchical representations, and hope that the first convolutional layer learns basic features (edges, simple curves, etc.), and the second convolutional layer will learn more complex features.
- Second convolutional layer is difficult to interpret even if decided to visualize it, so only include the first layer filters in Figure 5-18.

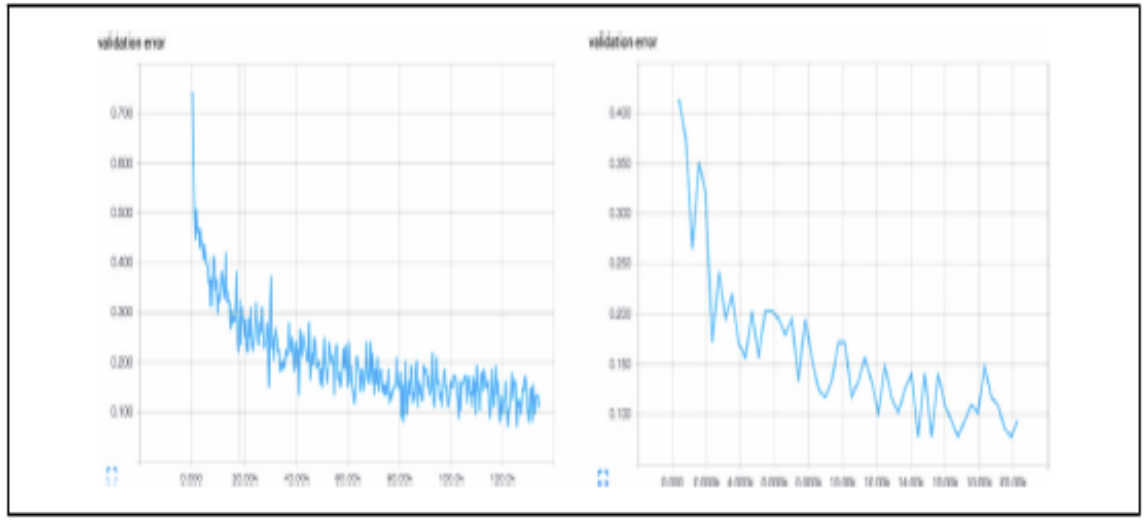


Figure 5-17. Training a convolutional network without batch normalization (left) versus with batch normalization (right). Batch normalization vastly accelerates the training process.

Figure 5-18. A subset of the learned filters in the first convolutional layer of our network

- Make out a number of interesting features in our filters: vertical, horizontal, and diagonal edges, in addition to small dots or splotches of one color surrounded by another.
- Network is learning relevant features because the filters are not just noise.
- To visualize how network has learned to cluster various kinds of images pictorially.
- Take a large network that has been trained on the ImageNet challenge and then grab the hidden state of the fully connected layer just before the softmax for each image.
- Take this high-dimensional representation for each image and use an algorithm known as *t-Distributed Stochastic Neighbor Embedding, or t-SNE, to compress it to a two-dimensional representation* can visualize.
- Don't cover the details of t-SNE here, but there are a number of publicly available software tools that will do it for us, including the script.
- Visualize the embeddings in Figure 5-19, and the results are quite spectacular.

Figure 5-19. The t-SNE embedding (center) surrounded by zoomed-in subsegments of the embedding (periphery). Image credit: Andrej Karpathy

- At first, on a high level, it seems that images that are similarly colored are closer together.
- When we zoom into parts of the visualization, we realize that it's more than just color.
- Realize that all pictures of boats are in one place, all pictures of humans are in another place, and all pictures of butterflies are in yet another location in the visualization.
- Convolutional networks have spectacular learning capabilities.

Leveraging Convolutional Filters to Replicate Artistic Styles

- Algorithms developed that leverage convolutional networks in much more creative ways.
- One of these algorithms is called *neural style.*
- *The goal of neural style is to be able to take an arbitrary photograph and rerender* it as if it were painted in the style of a famous artist.
- Clever manipulation of convolutional filters can produce spectacular results on this problem.

- Take a pre-trained convolutional network.
- There are three images that we're dealing with.
- The first two are the source of content ***p*** and the source of style ***a.***
- The third image is the generated image ***x.***
- Goal will be to derive an error function that can backpropagate that, when minimized, will perfectly combine the content of the desired photograph and the style of the desired artwork.
- Start with content first.
- If a layer in the network has *kl filters, then it produces a* total of *kl feature maps.*
- *Let's call the size of each feature map ml, the height times the* width of the feature map.
- The activations in all the feature maps of this layer can be stored in a matrix ***F(l) of size kl × ml.***
- Represent all the activations of the photograph in a matrix ***P(l)*** and all the ***activations*** of the generated image in the matrix ***X(l).***
- Use the ***relu4_2*** of the original ***VGGNet:***

$$E_{\text{content}}(\boldsymbol{p}, \boldsymbol{x}) = \Sigma_{ij}(\boldsymbol{P}_{ij}^{(l)} - \boldsymbol{X}_{ij}^{(l)})^2$$

- Construct a matrix known as the *Gram matrix, which represents correlations between feature maps in a given layer.*
- *The correlations* represent the texture and feel that is common among all features, irrespective of which features we're looking at.
- Constructing the Gram matrix, which is of size *kl × kl, for a given image is done as follows:* $\mathbf{G}^{(l)}_{ij} = \sum_{c=0}^{m_l} \mathbf{F}^{(l)}_{ic} \mathbf{F}^{(l)}_{jc}$
- Compute the Gram matrices for both the artwork in matrix ***A(l)*** and the generated image in ***G(l).***
- Represent the error function as: $E_{style}(\mathbf{a}, \mathbf{x}) = \frac{1}{4k_l^2 m_l^2} \Sigma_{l=1}^{L} \Sigma_{ij} \frac{1}{L} \left(A_{ij}^{(l)} - G_{ij}^{(l)}\right)^2$
- Weight each squared difference equally (dividing by the number of layers to include in our style reconstruction). Specifically, we use the relu1_1, relu2_1, relu3_1, relu4_1, and relu5_1 layers of the original VGGNet.
- Omit full a discussion of the TensorFlow code (*http://bit.ly/2qAODnp) for brevity, but the* results, as shown in Figure 5-20, are again quite spectacular.
- Mix a photograph of the iconic MIT dome and Leonid Afremov's *Rain Princess.*

Figure 5-20. The result of mixing the Rain Princess with a photograph of the MIT Dome. Image credit: Anish Athalye.

Learning Convolutional Filters for Other Problem Domains

- Natural extension of image analysis is video analysis.
- Using five-dimensional tensors (including time as a dimension) and applying three-dimensional convolutions is an easy way to extend the convolutional paradigm to video.
- Convolutional filters have also been successfully used to analyze audiograms.
- In these applications, a convolutional network slides over an audiogram input to predict phonemes on the other side.
- Less intuitively, convolutional networks have also found some use in natural language processing.
- More exotic uses of convolutional networks include teach algorithms to play board games, and analyzing biological molecules for drug discovery.